Machine Learning for Temperature Forecasting

Giorgio Bonetta (25-325-200)
Dawid Dymarczyk (17-293-796)
Radmehr Karimian (25-336-934)

Machine Learning
Geneva School of Economics and Management
University of Geneva

Lecturer: Sebastian Engelke
Teaching Assistants: Manuel Hentschel, Christophe Valvason

Fall 2025

Contents

1 Introduction **4**
1.1 Dataset Description . . . 4

2 Methodological Challenges **5**
2.1 Spatial Generalization and Extrapolation . . . 5
2.2 Temporal Structure under Forecasting Constraints . . . 6
2.3 Data Quality, Noise, and Robustness . . . 6
2.4 Overview of the Approach . . . 7
2.4.1 Unified evaluation protocol . . . 7
2.5 Data Preprocessing and Feature Engineering . . . 8
2.5.1 Preliminary Data Checks and Target Analysis . . . 8
2.5.2 Leakage Prevention and Feature Alignment . . . 8
2.5.3 Additional Preprocessing Details . . . 9
2.5.4 Zero-Inflation and Regime Decomposition . . . 10
2.6 Spatial Difference Features . . . 11
2.6.1 Transformation Plan and Reproducibility . . . 11
2.6.2 Empirical Implications . . . 11

3 Linear Models **12**
3.1 Data, Preprocessing, and Evaluation Protocol . . . 13
3.2 Baseline Predictors . . . 13
3.3 Experiment I: Classical Linear Models . . . 13
3.4 Experiment II: Robust Linear Models (MAE-aligned) . . . 15
3.5 Experiment III: Hyperparameter Tuning of Huber Regression . . . 15
3.6 Refitting on the Full Dataset and Coefficient Analysis . . . 16

4 Tree-Based Ensemble Models: Random Forest and Gradient Boosting **17**
4.1 Experimental Setup and Preprocessing . . . 18
4.2 Random Forest and Extra Trees Models . . . 18
4.3 Histogram Gradient Boosting Regression . . . 19
4.4 Model-Dependent Impact of Feature Engineering . . . 19
4.5 Random Forest Hyperparameter Tuning via Randomized Search . . . 20
4.5.1 Experimental Setup . . . 20
4.5.2 Results of Randomized Tuning . . . 21
4.6 Additional Methods: Gradient Boosting Baselines, Blending, and Uncertainty Estimation . 22
4.6.1 Preprocessing Strategy for Linear vs. Tree Models . . . 22
4.6.2 Experiment: Cross-Validated Comparison of Additional Models . . . 22
4.6.3 Blending via Out-of-Fold Predictions . . . 23
4.6.4 Quantile LightGBM for Uncertainty Bands . . . 23
4.7 Single Holdout Evaluation with Multiple Random Seeds . . . 24
4.7.1 Data and Leakage-Safe Preprocessing . . . 24
4.7.2 Robust Stratification for a Regression Target . . . 24
4.7.3 Model Configuration and Early Stopping . . . 24
4.7.4 Experiment: Five Seeds for Holdout Stability . . . 25
4.7.5 Final Training and Submission Export . . . 25
4.8 Two-Stage Hyperparameter Tuning of LightGBM with Random Search (Stage A/B) . . . 26
4.8.1 Dataset and Preprocessing . . . 26
4.8.2 Cross-Validated MAE with Early Stopping . . . 26
4.8.3 Resumable Experiment Design . . . 26
4.8.4 Stage A: Coarse Random Search . . . 27
4.8.5 Stage B: Local Refinement Around the Stage A Best . . . 27
4.8.6 Best Configuration Identified (Stage A/B Result) . . . 27

4.8.7 Final Training . . . 28
4.8.8 Implementation Notes . . . 28
4.9 Multi-Horizon Chaining and Blended Ensembling (12h/24h/48h) . . . 29
4.9.1 Predict 24h Directly and Indirectly via Deltas . . . 29
4.9.2 Data Alignment and Target Availability . . . 29
4.9.3 Leakage-Safe OOF Predictions for Each Sub-Model . . . 30
4.9.4 Tuning the Chaining Weights . . . 30
4.9.5 Tuning the Final 3-Way Blend . . . 31
4.9.6 Final Training and Submission Export . . . 31
4.10 Neural Network Baselines: MLP and 1D-CNN on Tabular Features . . . 33
4.10.1 Leakage-Safe Preprocessing and Cross-Validation Protocol . . . 33
4.10.2 MLP Architecture and Training . . . 33
4.10.3 Tabular 1D-CNN Architecture and Training . . . 33
4.10.4 Why Gradient Boosting Performs Better . . . 34

5 Conclusion **35**

A Use of Generative AI **36**

B Acknowledgements **36**

S403011 Machine Learning Project: Weather Prediction Team: DaGiRa

Authors: Giorgio Bonetta (25-325-200), Dawid Dymarczyk (17-293-796), Radmehr Karimian (25-336-934)

1 Introduction

A major reason why Switzerland has posed challenges in terms of forecasting temperature is due to the presence of complicated terrain and highly varying weather. The Alpine ranges result in pronounced geographical variation and microclimates, which cause large temperature variations over small areas. These aspects make traditional approaches unsuitable and result in the increasingly popular use of data-driven machine learning algorithms.

In our study, we specifically set our sights on forecasting air temperatures in Bern 24 hours into the future using machine learning methods trained and tested on data from a series of Swiss weather stations. While it is certainly conceivable that we could consider the problem of forecasting a time series problem and develop a machine learning system that predicts the next value in the series 24 hours into the future, we will take a more general approach by framing the problem as a supervised machine learning problem. Short-term forecasting of temperatures has numerous practical applications that make this challenge highly relevant, such as energy resource planning, transportation planning and agriculture. The codes are available here here.

1.1 Dataset Description

The dataset considered in this study is sourced from the Kaggle competition S-403011 ML UNIGE Weather Prediction and consists of hourly meteorological observations collected from a network of weather stations across Switzerland. The training dataset aggregates measurements from these ten stations; Basel, Genève, Interlaken, Sion, St. Gallen, Davos, Zermatt, Andermatt, La Dôle, and Lugano covering a wide range of climatic and topographical conditions, from a urban environments to Alpine regions. Each hourly observation comprises a rich set of atmospheric variables describing complementary physical processes relevant for temperature dynamics, related to wind, radiations, atmospheric pressure, precipitation and air temperature. A detailed summary of the variables included in the dataset, together with their corresponding data types, is reported in Table 1.1.

The training set, provided in `train.csv`, consists of $7{,}579$ hourly observations and 92 input features, together with target variables corresponding to future air temperature in Bern at multiple forecast horizons. Specifically, the dataset includes targets at +12, +24, and +48 hours. The presence of multiple forecast horizons in the training data allows for a consistent experimental design and provides flexibility for potential extensions to multi-horizon prediction settings. The test set, provided in `test.csv`, contains $3{,}247$ hourly observations associated exclusively with the Bern station. In contrast to the training data, target values are not available in the test set, which is used solely for out-of-sample evaluation and submission generation. Since Bern is not included among the ten training stations, this implies

Variable	Code	Data type
Wind speed (scalar)	`fkl010h0`	Numerical (continuous)
Wind gust peak	`fkl010h3`	Numerical (continuous)
Global radiation	`gre000h0`	Numerical (continuous)
Sunshine duration	`sre000h0`	Numerical (continuous)
Atmospheric pressure (sea level)	`pp0qffh0`	Numerical (continuous)
Atmospheric pressure (station level)	`prestah0`	Numerical (continuous)
Precipitation	`rre150h0`	Numerical (continuous)
Air temperature (2 m)	`tre200h0`	Numerical (continuous)
Lagged air temperature (2 m)	`tre200h0_lag24h`	Numerical (continuous)
Relative humidity (2 m)	`ure200h0`	Numerical (continuous)
Station identifier	Station suffix	Categorical (nominal)
Timestamp	`reference_timestamp`	Temporal (datetime)
Target temperature (+12h)	`target_tre200h0_plus12h`	Target (numerical, continuous)
Target temperature (+24h)	`target_tre200h0_plus24h`	Target (numerical, continuous)
Target temperature (+48h)	`target_tre200h0_plus48h`	Target (numerical, continuous)

Table 1.1: Summary of variables included in the dataset

a spatial generalization from the training station network to an unseen location. Thus, model performance relies not only on capturing temporal dependencies, like diurnal and synoptic- scale patterns, and also on gaining knowledge of the relationships between stations that persist despite geographical and topographical variation.

2 Methodological Challenges

Applying machine learning methods to short-term temperature forecasting in Switzerland entails many methodological challenges. These arise from the interaction between spatial heterogeneity, constrained temporal structure, and data quality issues typical of real-world meteorological observations. This section outlines the key challenges relevant to the proposed task and motivates the methodological choices adopted in the subsequent analysis.

2.1 Spatial Generalization and Extrapolation

A central difficulty of the forecasting problem considered in this study is the need to generalize spatially from observed stations to an unseen target location. Meteorological variables exhibit pronounced spatial heterogeneity due to differences in altitude, local topography, exposure, and surrounding terrain. As a result, both the marginal distributions of temperature and the relationships between atmospheric variables vary substantially across stations. This heterogeneity limits the effectiveness of models that implicitly assume station-level homogeneity and increases the risk of overfitting to location-specific patterns.

To address this challenge, the modeling strategy exploits information from all available stations simultaneously, encouraging the learning of cross-station relationships that are transferable to the target location. In particular, features derived from multiple stations and relative representations, such as spatial temperature differences, are designed to emphasize gradients and shared atmospheric structure rather than absolute levels tied to individual microclimates. Models with flexible inductive biases, especially tree-based ensembles, are well suited to this setting as they can accommodate nonlinear interactions and heterogeneous feature effects across space. We emphasize that, at prediction time, contemporaneous meteorological observations at the target location (Bern) are assumed to be available. The forecasting task therefore consists of predicting the air temperature in Bern at time $t + 24$ conditional on information observed up to time t, including current local measurements. Under this operational assumption, the target station is unseen only in the training phase with respect to its future values, while its current state is observable at inference time. This setting reflects realistic short-term forecasting scenarios in which present-time sensor readings are available.

2.2 Temporal Structure under Forecasting Constraints

Given the structure of the available data, we deliberately adopt a tabular supervised learning formulation, treating each observation as a snapshot of the atmospheric state at time t and mapping it to a 24-hour-ahead temperature outcome. Temporal structure is incorporated through carefully designed features rather than through explicit sequence modeling. Lagged temperature variables capture short-term persistence, while cyclical indicators such as hour of day and season encode seasonal effects. This approach allows models to exploit temporal regularities without relying on recurrent architectures or autoregressive training schemes, which would be incompatible with the available data structure. At the evaluation stage, particular attention is paid to avoiding information leakage arising from implicit temporal dependencies. Validation procedures are designed to ensure that preprocessing and model estimation do not exploit information unavailable at prediction time, thereby yielding realistic performance estimates.

2.3 Data Quality, Noise, and Robustness

The next problem arises from data quality inherent in observation using sensors, which include missing data, observation noise, and the presence of intense weather patterns that cause heavy tails and outliers in data distributions.

The missing data occurs because of a disruption of sensors or a problem with data transmission, and it has to be treated in a careful way such that it does not introduce any structure where none exists. Measurement noise and outlier data, despite the possibility that they may cause instabilities, contain meaningful information about atmospheric processes and cannot be arbitrarily discarded.

In the context of modeling, these aspects trigger the need to apply strong preprocessing techniques and loss functions, which do not place much emphasis on large deviations. In this case, assessing models involving the Mean Absolute Error helps address the issue by providing an evaluation measure in terms of degrees Celsius, hence dealing with large deviations. In this case, ensemble models reduce the effects of noise by combining predictions from numerous weak models.

These factors together form the methodological environment of the study, which shapes the rationale of the next sections on the design of the features.

2.4 Overview of the Approach

To counter the challenges identified in Section 2 with respect to methodological design, we propose a structured model-building process, where we emphasize data preprocessing, physically inspired feature construction, and a systematic model comparison with gradually increased capacity. The overall strategy is to provide a balance between interpretability, robustness, and fitting performance, within the limits allowed by the data type.

The pipeline has three major steps. First, the raw meteorological data are converted to a statistically stable and informative feature representation. Here, the goal is to counteract issues of raw data quality while placing a strict information-leakage-prevention requirement. Other preprocessing steps are ensured to be completely learned from the training data. As a result, preprocessing of test data is ensured to follow the same method. Second, the hierarchy of predictive models is assessed. A series of linear regression models, ranging from simple to robust, are used as benchmarks to ascertain the level of predictability that exists within the created space. These models form a foundation to set a baseline, which can then be improved upon. Following the baseline assessment, more flexible nonlinear models are presented, specifically tree-based ensemble methods, which are very efficient at determining nonlinear associations. Third, the performance of these models is evaluated within a single framework through cross-validation and Mean Absolute Error (MAE). Hyperparameters are tuned using either cross-validation or informed tuning, and the models will be refitted on the entire training data before making predictions on the test data.

Such an approach allows a clear comparison of the models, breaks the effect of feature engineering, and allows a logical judgment of the benefits of increasing the complexity of the models in the space-time heterogeneous forecasting environment.

2.4.1 Unified evaluation protocol

Unless otherwise stated, all performance metrics reported in this study are computed using 5-fold cross-validation with shuffled splits and a fixed random seed. This protocol serves as the primary estimator of generalization error and is applied consistently across all model classes to ensure fair comparison.

Mean Absolute Error (MAE) is used as the evaluation metric. All preprocessing steps, including imputation, encoding, and feature transformations, are fitted exclusively on the training portion of each fold and subsequently applied to the corresponding validation fold, ensuring strict leakage prevention.

Single hold-out validation splits (e.g. an 80/20 train–validation split) are employed solely for diagnostic purposes, such as early-stopping calibration or computationally intensive hyperparameter exploration, and are not used for final cross-model performance comparisons.

2.5 Data Preprocessing and Feature Engineering

Data preprocessing and feature engineering play a central role in the proposed modeling pipeline, as the statistical properties and representation of the input variables critically affect predictive performance and model stability. Given the heterogeneous nature of meteorological data and the lack of a fully exploitable temporal ordering, particular care is devoted to constructing a feature space that is statistically well-behaved, physically interpretable, and robust to overfitting. All preprocessing choices are informed by empirical diagnostics computed exclusively on the training data and are applied identically to the test set to prevent information leakage.

2.5.1 Preliminary Data Checks and Target Analysis

Prior to any feature transformation, a set of preliminary sanity checks was performed to verify the structural integrity of the dataset. Duplicate observations were removed, and variables exhibiting zero or near-zero variance were identified, as such features provide negligible predictive information and may adversely affect model stability. Figure 2.1 reports the empirical distribution of the target(air temperature in Bern 24 hours ahead), which exhibits mild skewness and the presence of occasional extreme values.

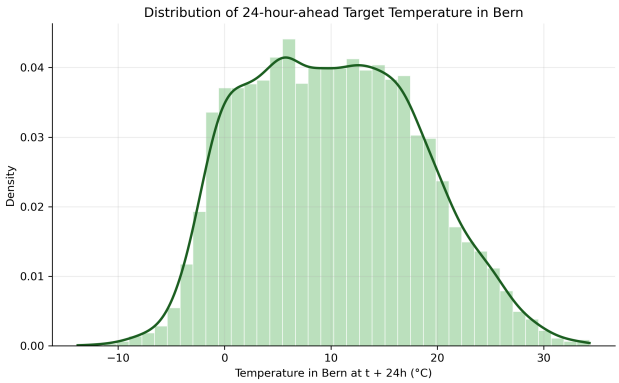

Figure 2.1: Empirical distribution of the 24-hour-ahead target temperature in Bern.

These distributional properties supports the adoption of robust loss functions, such as the Huber loss, which balance sensitivity to typical errors with resistance to outliers.

2.5.2 Leakage Prevention and Feature Alignment

Preventing information leakage is essential for obtaining reliable performance estimates. All auxiliary target variables corresponding to alternative forecast horizons were removed from the feature set, as they contain future information relative to the prediction task. Identifier columns used solely for indexing observations were also excluded. Strict feature alignment between the training and test datasets was enforced by retaining only the intersection of available features. This guarantees that models are trained and evaluated on identical feature spaces and prevents discrepancies at prediction time.

2.5.3 Additional Preprocessing Details

Additional preprocessing steps include median imputation for numerical variables, mode imputation for categorical variables, and model-aware standardization applied exclusively to linear models. Variables expressed in physically linear units (e.g. temperature and pressure) are retained on their original scale to preserve interpretability.

A deterministic, rule-based transformation plan is defined based solely on training-set diagnostics and applied consistently across validation and test data. These implementation details are designed to ensure robustness, reproducibility, and leakage prevention, while remaining secondary to the core methodological contributions discussed above. To guide feature transformation in a principled and interpretable manner, predictors were grouped into broad semantic categories using name-based heuristics. These categories include temperature-related variables, precipitation and radiation fluxes, wind variables, atmospheric pressure variables, humidity variables, binary indicators, and a residual group of uncategorized features. This lightweight taxonomy provides an approximate characterization of the physical meaning of each variable and enables category-specific diagnostic and transformation rules without relying on external metadata.

For each feature, a set of diagnostic statistics was computed exclusively on the training data. These diagnostics include the proportion of missing values, the share of zero observations, the presence of negative values, empirical skewness, numerical range, and the number of unique values. Restricting all diagnostics to the training set ensures that subsequent transformation decisions do not depend on information from the test distribution, thereby preserving the integrity of the evaluation. In addition,

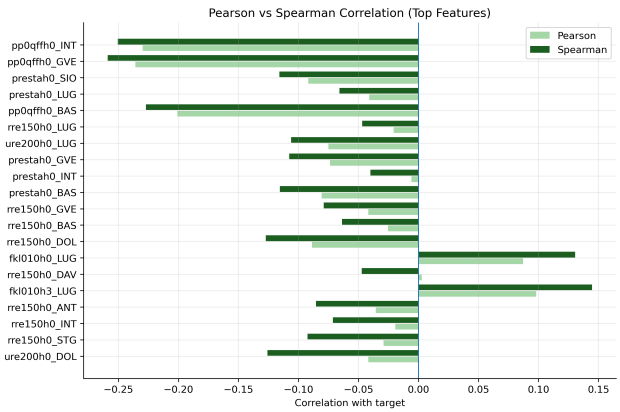

Figure 2.2: Comparison between Pearson and Spearman correlations for selected features.

both Pearson and Spearman correlation coefficients were computed to assess the relationship between each predictor and the target variable. While Pearson correlation captures linear dependence, Spearman correlation is sensitive to more general monotonic relationships. Figure 2.2 illustrates that several features exhibit substantially stronger rank-based than linear correlations with the target, providing empirical support for the use of nonlinear models, such as tree-based ensembles and gradient boosting methods. Several accumulation and flux variables, particularly those related to precipitation and radiation, exhibit strong right skewness and heavy-tailed behavior. Using such variables on their

original scale can lead to unstable estimates and excessive sensitivity to rare extreme events.

For strictly non-negative variables with pronounced skewness, a logarithmic transformation of the form $x \mapsto \log(1+x)$ was applied. Variables with moderate skewness were instead transformed using a square-root mapping, which provides a gentler adjustment while preserving more of the original scale. Variables that can assume negative values, such as wind components or signed fluxes, were left untransformed to preserve physical interpretability.

2.5.4 Zero-Inflation and Regime Decomposition

Several meteorological variables exhibit pronounced zero inflation, taking the value zero for a large fraction of observations and becoming positive only under specific atmospheric conditions, such as precipitation or radiation events. This distributional pattern reflects an underlying regime-switching behavior, in which both the occurrence and the intensity of an event carry predictive information.

To capture this structure, zero-inflated variables are decomposed into two complementary components: a binary indicator encoding event occurrence and a transformed magnitude feature describing intensity conditional on occurrence. This decomposition enables the model to distinguish between the probability of an event and its conditional impact on the target variable.

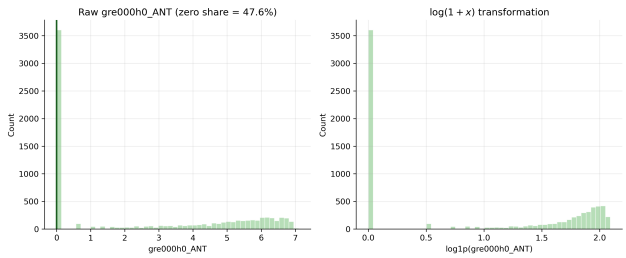

Figure 2.3: Effect of a $\log(1+x)$ transformation on a zero-inflated precipitation variable.

As illustrated in Figure 2.3, the logarithmic transformation applied to positive observations compresses extreme values and stabilizes variance, while the binary indicator preserves information about regime changes.

Variables representing temperature, atmospheric pressure, or geopotential height are retained on their original scale. These quantities are expressed in physically meaningful units and typically exhibit approximately linear relationships with the target variable; nonlinear transformations did not yield consistent empirical improvements and would reduce interpretability without clear benefits.

Categorical variables, such as seasonal indicators, are encoded using one-hot encoding after imputing missing values with the most frequent category. The encoding procedure is configured to ignore unseen categories at prediction time, ensuring robustness when applying models to new data.

Preprocessing strategies are adapted to the model class under consideration. Numerical features are standardized for linear models to ensure stable optimization and interpretable regularization, while no scaling is applied for tree-based and boosting models, which are invariant to monotonic transformations.

All spatial features are constructed exclusively from information available at time t and are therefore

strictly leakage-safe with respect to the 24-hour forecasting horizon. No future values of the target variable or auxiliary targets are used at any stage of feature construction.

2.6 Spatial Difference Features

To enhance spatial generalization to the unseen target location, additional features are constructed by computing differences between contemporaneous temperature measurements at surrounding stations and the air temperature observed at the target location (Bern) at time t. Formally, for each auxiliary station $s \neq$ Bern, we define

$$\Delta T_{s,t} = T_{s,t} - T_{\text{Bern},t},$$

where all quantities are observed at the same reference time t. These spatial difference features encode relative temperature gradients rather than absolute levels, allowing the model to focus on transferable spatial structure while remaining invariant to station-specific climatological offsets. Importantly, since $T_{\text{Bern},t}$ is observed at prediction time, the construction of $\Delta T_{s,t}$ does not introduce information leakage with respect to the forecasting horizon $t + 24$.

2.6.1 Transformation Plan and Reproducibility

Based on feature diagnostics and semantic categories, a deterministic, rule-based transformation plan was defined and stored in a structured format. All transformations were learned exclusively from the training data and applied identically to the test set. Newly created features were explicitly labeled to preserve traceability. Separating feature construction from model training improves reproducibility, simplifies experimentation, and ensures that all models are evaluated on an identical feature representation.

2.6.2 Empirical Implications

The empirical results reported in Tables 2.1–2.3 show that the impact of feature engineering is strongly model-dependent. Linear models and bagging-based tree ensembles exhibit limited or negative sensitivity to engineered features, while boosting-based methods benefit consistently from distribution-stabilizing transformations and spatial difference predictors.

Model	**RAW MAE**	**RAW STD**	**FE MAE**	**Δ (RAW–FE)**
OLS (Linear Regression)	1.98846	0.01326	1.99285	-0.00439
Ridge Regression ($\alpha = 10$)	1.98324	0.01266	1.98798	-0.00474
LAD Regression (Quantile, $q = 0.5$, $\alpha = 0.1$)	2.16371	0.01950	2.16371	0.00000

Table 2.1: Effect of feature engineering on linear models.

Table 2.1 summarizes the effect of feature engineering on linear models. Across ordinary least squares, Ridge regression, and LAD regression, engineered features do not lead to systematic improvements in MAE, reflecting the limited expressiveness of linear models and their inability to exploit nonlinear transformations or interaction proxies.

Model	RAW MAE	FE MAE	Δ (RAW–FE)
RF (400 trees, depth 15)	1.89423	1.89863	-0.00440
RF (600 trees, depth 20)	1.87668	1.87678	-0.00011
ET (600 trees, depth 15)	1.94891	1.95300	-0.00408
ET (800 trees, depth 20)	1.89082	1.89880	-0.00798

Table 2.2: Effect of feature engineering on Random Forest and Extra Trees models.

As shown in Table 2.2, feature engineering results in small but consistent performance degradation across Random Forest and Extra Trees configurations. Bagging-based tree ensembles already perform implicit feature selection and are largely invariant to monotonic transformations, such that additional engineered features primarily increase dimensionality without introducing new informative split signals.

Model	RAW MAE	FE MAE	Δ (RAW–FE)
HGBR (MAE, depth 6, 600 iterations)	1.84291	1.83984	$+0.00307$

Table 2.3: Effect of feature engineering on Histogram Gradient Boosting.

In contrast, Table 2.3 indicates a consistent, albeit modest, improvement from feature engineering for Histogram Gradient Boosting Regression. By iteratively refining residual structure, boosting-based models can exploit transformed predictors and difference-based features to improve split quality, particularly for skewed and zero-inflated variables.

These results combined indicate that feature engineering does not uniformly enhance predictive performance but strongly interacts with model inductive bias. Linear and bagging-based models derive little benefit from the contrived feature space, whereas boosting-based methods directly exploit The proposed transformations suggest that feature engineering is acting as a Complementary mechanism enhancing model capacity rather than substituting it. for it.

3 Linear Models

We begin the modeling analysis with classical linear regression methods, which provide an interpretable and statistically well-understood baseline for short-term temperature forecasting. Although linear models are not designed to capture nonlinear, spatially heterogeneous, and regime-dependent dynamics of atmospheric processes, they remain valuable because they quantify how much predictive performance can be achieved through additive relationships alone and establish a transparent benchmark against which more flexible machine learning models can be evaluated. Since explicit timestamps are unavailable, all models operate on snapshot-based predictors. Consequently, predictive performance reflects the instantaneous information content encoded in the feature representation rather than explicit temporal dynamics.

3.1 Data, Preprocessing, and Evaluation Protocol

We applied the preprocessing and feature engineering pipeline described in Section 2.5. Table 3.1 summarizes the dataset composition and the evaluation setup used throughout the linear experiments.

Item	Value
Training observations (raw)	7,579
Training observations (used)	7,578
Dropped observations	1 (missing target)
Number of features	109
Numerical features	108
Categorical features	1 (`season`)
Target variable	`target_tre200h0_plus24h`
Baseline predictor	Target median
Baseline MAE	6.72175
Cross-validation scheme	5-fold K-Fold (shuffled)
Evaluation metric	Mean Absolute Error (MAE)

Table 3.1: Setup for linear models.

Model performance is evaluated using 5-fold cross-validation with shuffled splits. All preprocessing steps, including imputation, encoding, and feature transformations, are fitted exclusively on the training folds and applied to the corresponding validation folds to prevent information leakage.

3.2 Baseline Predictors

Asides the general median predictor, another is a simple physics-inspired persistence baseline. According to this approach, the air temperature 24 hours ahead was predicted to be equivalent to the contemporary observed value. temperature,

$$\hat{T}_{t+24} = T_t.$$

This baseline integrates the high level of autocorrelation and day-night cycles: structure that is characteristic of near-surface air temperatures and offers a physically important point of comparison in evaluating the value added through machine learning models.

3.3 Experiment I: Classical Linear Models

The first experiment evaluated standard linear regression models that minimize squared error, with and without regularization. Ordinary least squares (OLS) regression provides maximum interpretability through linear coefficients and achieved a cross-validated MAE of 1.993, demonstrating that residuals are reasonably well-behaved after preprocessing even though OLS optimizes a squared-error loss rather than MAE. Given the strong multicollinearity expected from spatially correlated weather stations and overlapping meteorological variables, Ridge regression was evaluated to stabilize estimation.

Ridge regression augments OLS with an ℓ_2 penalty,

$$\min_{\beta} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \beta)^2 + \lambda \|\beta\|_2^2,$$

and moderate regularization improved predictive performance. The best classical linear result was obtained for Ridge regression with $\alpha = 1.0$ (MAE = 1.986), while strong regularization ($\alpha = 100$) degraded performance, indicating underfitting. Lasso regression introduces an ℓ_1 penalty and Elastic Net combines ℓ_1 and ℓ_2 penalties. Both underperformed Ridge regression in this application, suggesting that predictive signal is spread across many correlated predictors rather than concentrated in a small subset, making shrinkage more effective than sparsity. Table 3.2 reports the full results of Experiment I. Moderate ℓ_2 regularization yields the best results among squared-error models, while overly strong penalization leads to underfitting. Sparsity-inducing penalties provide no advantage in this setting, suggesting that predictive signal is distributed across many correlated predictors.

Model	**CV MAE (mean)**	**CV MAE (std)**
Ridge ($\alpha = 1.0$)	1.98598	0.01253
Lasso ($\alpha = 0.001$)	1.98607	0.01065
Ridge ($\alpha = 10.0$)	1.98798	0.01130
OLS	1.99285	0.01740
Elastic Net ($\alpha = 0.01$, $l_1 = 0.5$)	1.99933	0.01649
Lasso ($\alpha = 0.01$)	2.00084	0.01681
Ridge ($\alpha = 100.0$)	2.00168	0.01683

Table 3.2: Experiment I: Cross-validated MAE for classical linear models.

Figure 3.1 provides a complementary visual summary of Experiment I. The plot highlights the improvement produced by moderate regularization relative to OLS and shows that overly strong penalization increases error. Error bars represent one standard deviation across folds.

Figure 3.1: Cross-validated MAE for classical linear models.

All linear models substantially outperform both the constant and persistence baselines, indicating that

spatial information from surrounding stations and additional atmospheric variables provide predictive value beyond simple temporal continuity.

3.4 Experiment II: Robust Linear Models (MAE-aligned)

The second experiment focused on linear models designed to be robust to outliers and directly aligned with MAE. Least absolute deviations (LAD) regression was evaluated via median quantile regression ($q = 0.5$),

$$\min_{\beta} \sum_{i=1}^{n} |y_i - \mathbf{x}_i^\top \beta|.$$

An unregularized LAD model ($\alpha = 0.0$) achieved a cross-validated MAE of 1.977, outperforming all squared-error-based linear models. However, LAD regression proved highly sensitive to regularization. Mild penalization ($\alpha = 0.1$) substantially degraded performance, and stronger regularization ($\alpha = 1.0$) caused collapse toward the median baseline. Huber regression provides a smooth compromise between squared-error and absolute-error losses. For small residuals the loss is quadratic, while for large residuals it grows linearly,

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta\left(|r| - \frac{1}{2}\delta\right), & |r| > \delta. \end{cases}$$

Table 3.3 reports the results of Experiment II. As reported in Table, robust linear models aligned with the MAE objective outperform squared-error baselines. With $\epsilon = 1.35$, Huber regression achieved the best overall linear performance (MAE = 1.971), indicating that the dataset benefits from limited robustness while preserving efficient learning around the bulk of the residual distribution.

Model	**CV MAE (mean)**	**CV MAE (std)**
Huber Regression ($\epsilon = 1.35$)	1.97095	0.01911
LAD ($q = 0.5$, $\alpha = 0.0$)	1.97740	0.02429
LAD ($q = 0.5$, $\alpha = 0.1$)	2.16371	0.01950
LAD ($q = 0.5$, $\alpha = 1.0$)	6.72264	0.11261

Table 3.3: Experiment II: Cross-validated MAE for robust linear models.

3.5 Experiment III: Hyperparameter Tuning of Huber Regression

After identifying Huber regression as the strongest linear baseline, a third experiment was conducted to systematically tune its hyperparameters. The robustness parameter ϵ and regularization strength α were evaluated over a grid using the same preprocessing pipeline and 5-fold CV protocol:

$$\epsilon \in \{1.05, 1.10, 1.20, 1.35, 1.50, 1.75, 2.00\}, \qquad \alpha \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}.$$

As shown in Table 3.4, the best configuration was $\epsilon = 1.35$ and $\alpha = 0.1$, yielding a mean MAE of 1.9706. Several nearby configurations achieved nearly identical performance, indicating a broad plateau of near-optimal solutions.

ϵ	α	**CV MAE**
1.35	0.10	1.97061
1.20	0.10	1.97138
1.05	0.10	1.97166
1.10	0.10	1.97169
1.50	0.10	1.97173

Table 3.4: Top-performing Huber regression configurations under 5-fold cross-validation.

Figure 3.2 visualizes the MAE surface across the (ϵ, α) grid and highlights the near-optimal plateau around moderate ϵ values.

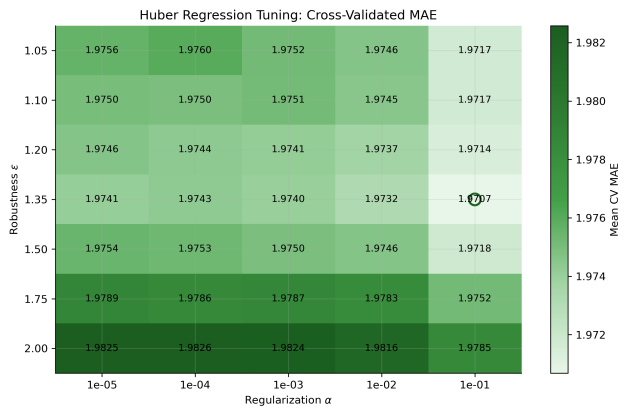

Figure 3.2: Cross-validated MAE for Huber regression across ϵ and α.

3.6 Refitting on the Full Dataset and Coefficient Analysis

To obtain stable coefficient estimates and enable interpretation, the selected Ridge regression model ($\alpha = 1.0$) was refitted on the full training dataset. Because numerical predictors were standardized, coefficient magnitudes are directly comparable across variables. Each coefficient can be interpreted as the expected change in the predicted 24-hour-ahead temperature (in degrees Celsius) associated with a one-standard-deviation increase in the predictor, holding other variables fixed. Seasonal dummy variables reflect climatological offsets, while the largest coefficients typically correspond to physically meaningful predictors such as pressure-related variables and near-surface temperatures at surrounding stations. Table 3.5 reports a subset of the largest-magnitude coefficients. The signs and magnitudes are consistent with meteorological intuition, indicating that regional pressure configurations and spatial coherence of temperature measurements provide strong signals for forecasting Bern.

Feature	Coefficient
pp0qffh0_BAS	-4.32
prestah0_INT	$+4.05$
pp0qffh0_GVE	-3.57
prestah0_ANT	-2.46
prestah0_SIO	$+2.16$
tre200h0 (Bern)	$+1.56$
season_Summer	$+0.90$
season_Winter	-0.68

Table 3.5: Selected coefficients of the Ridge regression model ($\alpha = 1.0$) after refitting on the full training set. Coefficients are reported after standardization.

Figure 3.3 provides a complementary visualization of the same information.

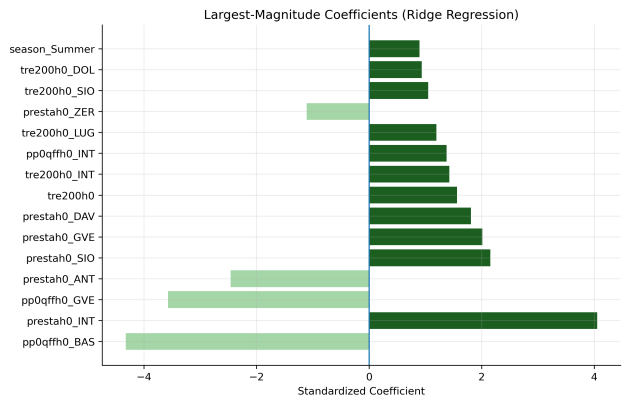

Figure 3.3: Largest-magnitude coefficients of the selected Ridge regression model after refitting on the full training set.

In sum, linear methods offer a strong baseline for this problem, indicating that a significant amount of predictive information is accessible through additive modeling. However, the existence of error and the value of robust loss functions imply that nonlinear processes are playing an important role. It is these deficiencies that form the motivation for moving towards nonlinear ensemble techniques in the following sections. Note the following about the results of the CSN analysis on the second level of the example hierarchical model the chosen regularization strategy and the form of multicollinearity, and cannot be interpreted as causal effects.

4 Tree-Based Ensemble Models: Random Forest and Gradient Boosting

Having established linear and bagging-based models as informative but ultimately limited baselines, we now turn to boosting-based approaches, which are specifically designed to capture nonlinear interactions and structured residual patterns. These models are particularly well suited for heterogeneous,

non-Gaussian data and can naturally represent interactions between predictors without explicit feature construction. In this section, we evaluate bagging-based ensembles (Random Forest and Extra Trees) as well as boosting-based ensembles, focusing on Histogram Gradient Boosting Regression (HGBR). All experiments in this section use the same target variable, `target_tre200h0_plus24h`, and are evaluated using mean absolute error (MAE) under a 5-fold shuffled cross-validation scheme with a fixed random seed. To assess the effect of feature engineering, each model is trained and evaluated on both the raw feature set and the feature-engineered (FE) dataset.

4.1 Experimental Setup and Preprocessing

Two input representations are compared. The raw dataset contains 89 predictors derived directly from meteorological measurements after removal of all target-related columns and identifier fields. The feature-engineered dataset contains 109 predictors, including nonlinear transformations, zero-inflation indicators, and difference-based features designed to expose latent structure while remaining leakage-safe. For all tree-based models, preprocessing is performed within each cross-validation fold using a `Pipeline`. Numerical predictors are median-imputed, while categorical variables are mode-imputed and one-hot encoded. No feature scaling is applied, as tree-based models are invariant to monotonic transformations of individual predictors. To control computational cost and reduce overfitting, all ensemble models are explicitly depth-limited and constrained through minimum leaf sizes. This design choice ensures that differences in performance can be attributed primarily to model class and feature representation rather than excessive model capacity.

4.2 Random Forest and Extra Trees Models

Random Forest (RF) and Extra Trees (ET) are bagging-based ensemble methods that average predictions from multiple decorrelated decision trees. Random Forest introduces randomness through bootstrap sampling and feature subsampling at each split, while Extra Trees further increase randomness by selecting split thresholds uniformly at random. Both methods are known for their robustness and low variance but are generally insensitive to monotonic feature transformations. Multiple configurations of RF and ET were evaluated, varying the number of trees, maximum depth, and minimum leaf size. All models used `max_features = sqrt`, a standard choice that balances bias and variance. Table 4.1 reports the cross-validated MAE for Random Forest and Extra Trees models trained on raw and feature-engineered data.

Model	**RAW MAE**	**RAW STD**	**FE MAE**	**Improvement (RAW–FE)**
RF (400 trees, depth 15, leaf 5)	1.89423	0.02207	1.89863	-0.00440
RF (600 trees, depth 20, leaf 3)	1.87668	0.02385	1.87678	-0.00011
ET (600 trees, depth 15, leaf 5)	1.94891	0.02419	1.95300	-0.00408
ET (800 trees, depth 20, leaf 3)	1.89082	0.02247	1.89880	-0.00798

Table 4.1: Effect of feature engineering on Random Forest and Extra Trees models. Lower MAE indicates better performance.

Across all Random Forest and Extra Trees configurations, feature engineering resulted in negligible or slightly negative performance changes. The observed MAE differences are small in magnitude but consistently unfavorable for the feature-engineered representation. This behavior is expected: bagging-based tree ensembles already perform implicit variable selection and partition the feature space via thresholding. As a result, monotonic transformations such as logarithmic or square-root scaling do not alter the ordering of values and therefore provide little additional information. Moreover, the expanded feature space introduced by feature engineering increases dimensionality without contributing new, strongly discriminative split signals, slightly increasing variance.

These results indicate that Random Forest and Extra Trees models are largely insensitive to the proposed feature engineering strategy and achieve their best performance when trained on the raw feature set.

4.3 Histogram Gradient Boosting Regression

In contrast to bagging-based ensembles, boosting-based methods iteratively fit new trees to the residuals of the current model. This sequential procedure allows boosting algorithms to exploit subtle patterns, interactions, and nonlinearities that may not be captured by individual trees alone. Histogram Gradient Boosting Regression (HGBR) is a particularly efficient implementation that discretizes continuous features into bins, significantly reducing computational cost while retaining strong predictive performance. An HGBR model was trained with an MAE-aligned loss function (`absolute_error`), a maximum tree depth of six, and 600 boosting iterations. These hyperparameters were chosen to balance model expressiveness and computational efficiency.

Table 4.2 reports the cross-validated MAE for HGBR on raw and feature-engineered data.

Model	**RAW MAE**	**RAW STD**	**FE MAE**	**Improvement (RAW–FE)**
HGBR (MAE, depth 6, 600 iters)	1.84291	0.01824	1.83984	$+0.00307$

Table 4.2: Effect of feature engineering on Histogram Gradient Boosting Regression.

Unlike Random Forest and Extra Trees, HGBR exhibits a consistent improvement when trained on the feature-engineered dataset. Although the absolute gain in MAE is modest, it is stable across cross-validation folds. This improvement suggests that boosting-based models are able to exploit the additional structure introduced by feature engineering, including transformed distributions, interaction proxies, and difference-based predictors. By iteratively refining residuals, HGBR can convert these engineered features into more informative splits, improving overall predictive accuracy.

4.4 Model-Dependent Impact of Feature Engineering

The experimental results demonstrate that the effectiveness of feature engineering is strongly model-dependent. Bagging-based tree ensembles, such as Random Forest and Extra Trees, derive little benefit from the engineered features and in some cases experience slight performance degradation. Their inherent robustness and invariance to monotonic transformations limit the utility of additional

feature manipulation.

In contrast, gradient boosting models benefit measurably from the engineered representation. The sequential nature of boosting enables these models to leverage transformed variables and interaction proxies more effectively, leading to improved residual structure and lower MAE. This distinction mirrors observations from the linear modeling experiments and reinforces the conclusion that feature engineering primarily benefits models capable of exploiting complex, nonlinear dependencies.

Overall, Histogram Gradient Boosting emerges as the strongest performer among the tree-based models evaluated in this section and provides a clear motivation for further exploration of boosting-based approaches, including more advanced gradient boosting frameworks and hybrid modeling strategies.

4.5 Random Forest Hyperparameter Tuning via Randomized Search

The previous Random Forest experiments demonstrated that, under reasonable depth and leaf-size constraints, Random Forest models achieve competitive but clearly suboptimal performance compared to boosting-based methods. Moreover, feature engineering did not improve Random Forest accuracy, suggesting that model capacity rather than representation might be the limiting factor. To verify whether stronger Random Forest configurations could close the performance gap, a dedicated hyperparameter tuning experiment was conducted on the feature-engineered dataset.

The objective of this experiment was not exhaustive optimization but rather to assess whether Random Forest performance could be meaningfully improved through more aggressive tuning while maintaining computational feasibility. Given the large size of the dataset and the high cost of training deep ensembles, a randomized search strategy was adopted.

4.5.1 Experimental Setup

The tuning experiment was conducted on the feature-engineered dataset consisting of 7,578 observations and 109 predictors. As in previous tree-based experiments, numerical variables were median-imputed and categorical variables were mode-imputed and one-hot encoded within a leakage-safe preprocessing pipeline. No feature scaling was applied. To reduce computational cost relative to earlier experiments, model evaluation was performed using 3-fold cross-validation with shuffled splits and a fixed random seed. Mean absolute error (MAE) was used as the evaluation metric, consistent with the competition objective and prior experiments. A bounded but expressive hyperparameter search space was defined, focusing on parameters known to control Random Forest bias–variance trade-offs:

$$\begin{aligned}
\text{n_estimators} &\in \{250, 400, 600, 800, 1000\},\\
\text{max_depth} &\in \{12, 16, 20, 30, 50\},\\
\text{min_samples_leaf} &\in \{1, 3, 5, 10\},\\
\text{min_samples_split} &\in \{2, 5, 10\},\\
\text{max_features} &\in \{\text{sqrt}, 0.5, 0.8\}.
\end{aligned}$$

All models were trained with bootstrap sampling enabled and parallelized across CPU cores. From this space, 100 hyperparameter configurations were sampled uniformly at random using `ParameterSampler`

and evaluated independently.

4.5.2 Results of Randomized Tuning

Table 4.3 reports the top-performing Random Forest configurations ranked by mean cross-validated MAE.

Trees	Depth	Leaf	Split	Max feat.	MAE	STD
800	30	1	2	sqrt	**1.88011**	0.01586
600	30	1	2	sqrt	1.88220	0.01615
1000	30	1	5	sqrt	1.88318	0.01518
400	20	1	2	sqrt	1.88501	0.01472
600	20	1	2	0.5	1.88676	0.01635

Table 4.3: Top Random Forest configurations from randomized hyperparameter tuning (3-fold CV, feature-engineered data).

The best-performing configuration achieved a mean MAE of 1.880, using 800 trees, a maximum depth of 30, a minimum leaf size of 1, and `max_features = sqrt`. This represents a modest improvement over the earlier bounded-depth Random Forest configurations, which achieved MAE values in the range of approximately 1.88–1.90.

Despite this improvement, even the strongest tuned Random Forest models remain substantially worse than boosting-based approaches evaluated elsewhere in this study, particularly Histogram Gradient Boosting and LightGBM, which achieved MAE values below 1.84 and, in later experiments, below 1.80. Increasing the number of trees and allowing deeper splits yields diminishing returns, while dramatically increasing training time. The full randomized search required over five hours of computation for 100 configurations, highlighting the limited efficiency of Random Forest tuning at this scale.

The randomized tuning experiment confirms that Random Forest performance can be improved slightly through careful hyperparameter selection, primarily by increasing tree depth and reducing leaf constraints. However, the magnitude of the improvement is small relative to the computational cost incurred. The results reinforce the conclusion that Random Forest models are fundamentally limited in their ability to exploit the structure of this problem, even when aggressively tuned.

This behavior is consistent with the theoretical properties of bagging-based ensembles. Random Forests reduce variance by averaging many deep trees but do not perform sequential error correction. As a result, they struggle to model fine-grained residual structure and complex interactions that are critical for high-accuracy short-term temperature forecasting.

Consequently, while Random Forest provides a strong and robust baseline, further performance gains are more effectively achieved by boosting-based methods rather than by additional Random Forest tuning. This observation motivates the subsequent focus on gradient boosting models and hybrid approaches.

4.6 Additional Methods: Gradient Boosting Baselines, Blending, and Uncertainty Estimation

After establishing baselines with linear models and bagging-based trees (Random Forest and Extra Trees), a complementary set of models was evaluated to (i) benchmark stronger gradient boosting learners, (ii) test whether combining a robust linear model with a nonlinear tree model improves accuracy, and (iii) obtain uncertainty estimates via quantile regression. All experiments in this subsection were performed on the feature-engineered dataset `X_train_prepared.csv` and the corresponding target vector `y_train.csv`. One observation with missing target was removed, leaving $n = 7578$ samples with $p = 109$ predictors.

4.6.1 Preprocessing Strategy for Linear vs. Tree Models

Two preprocessing pipelines were used, each embedded inside cross-validation folds to avoid leakage. For tree-based models, numerical features were median-imputed and categorical features were mode-imputed and one-hot encoded, without any scaling. For linear models, the same imputations and one-hot encoding were applied, but numerical variables were additionally standardized to zero mean and unit variance. This separation is motivated by the different inductive biases of the model classes: linear methods depend strongly on feature scale, while tree learners are generally scale-invariant.

4.6.2 Experiment: Cross-Validated Comparison of Additional Models

The following models were evaluated using 5-fold K-Fold cross-validation (shuffled, fixed seed) with mean absolute error (MAE) as the metric. A constant median predictor was included as a minimal benchmark, since the median minimizes MAE among constant models. The evaluated methods were Huber regression (robust linear baseline), LightGBM regression with an MAE-aligned objective (`regression_l1`), and XGBoost regression with absolute-error objective (`reg:absoluteerror`). CatBoost was included in the code as a candidate because it can handle categorical variables natively; however, it was not available in the runtime environment for this experiment.

Table 4.4 reports the resulting MAE scores.

Model	CV MAE (mean)	CV MAE (std)
LightGBM (`regression_l1`)	**1.79396**	0.01541
XGBoost (`reg:absoluteerror`)	1.82692	0.01473
Huber ($\epsilon = 1.35$, $\alpha = 10^{-3}$)	1.97370	0.02248
Baseline (median)	6.72175	–

Table 4.4: Cross-validated MAE for additional methods on the feature-engineered dataset ($n = 7578$). Lower is better.

The results confirm that boosting-based tree models substantially outperform linear baselines. LightGBM achieves the strongest performance among the tested models with an MAE of 1.794, followed by XGBoost with an MAE of 1.827. Both methods improve markedly over Huber regression, indicating

that nonlinear interaction structure and threshold effects are important for this forecasting task. The proximity of LightGBM and XGBoost suggests that both frameworks are competitive, while the performance gap between boosting and linear methods reflects the limitation of additive linear structure under complex atmospheric dynamics.

4.6.3 Blending via Out-of-Fold Predictions

To test whether a robust linear learner contributes complementary information to the nonlinear tree model, a simple blending experiment was conducted between Huber regression and the best available tree model (LightGBM in this setting). The blend was constructed using out-of-fold (OOF) predictions. Specifically, each model was trained inside each fold, predictions were collected on the held-out fold to form unbiased OOF estimates, and then a scalar weight $w \in [0, 1]$ was chosen to minimize MAE of the convex combination:

$$\hat{y}_{\text{blend}} = w\hat{y}_{\text{Huber}} + (1 - w)\hat{y}_{\text{Tree}}.$$

A simple grid search over w (201 values) identified the best weight. The optimal blend assigned a relatively small weight to Huber regression,

$$w = 0.110, \qquad \text{MAE}_{\text{OOF blend}} = 1.79105.$$

This blended MAE is slightly better than the standalone LightGBM result (1.79396), indicating that the robust linear model contributes a small but nonzero complementary signal. In practical terms, the improvement is modest, but it demonstrates that even when a boosted tree dominates, an additional learner with different inductive bias can reduce error through ensembling. The blending diagnostics were saved to `blend_oof_predictions.csv`. The cross-validated leaderboard was saved to `extra_methods_cv_results.csv`.

4.6.4 Quantile LightGBM for Uncertainty Bands

Point predictions alone do not quantify uncertainty. To provide uncertainty information, quantile regression models were fitted using LightGBM with `objective="quantile"` and quantiles $q \in \{0.1, 0.5, 0.9\}$. This produces conditional quantile estimates $\hat{y}_q(x)$, which can be interpreted as an empirical prediction interval: $[\hat{y}_{0.1}(x), \hat{y}_{0.9}(x)]$, with $\hat{y}_{0.5}(x)$ serving as a median prediction.

Unlike the previous experiments, these quantile models were trained on the full dataset without cross-validation, since the goal was to generate full-sample quantile outputs rather than compare models under a CV protocol.

The resulting quantile predictions on the training data were saved to `quantile_intervals_on_train.csv`.

During this quantile fitting step, LightGBM produced warnings of the form “No further splits with positive gain, best gain: -inf”. This behavior is common when certain boosting iterations cannot find a beneficial split under the chosen objective and regularization constraints. It does not necessarily indicate model failure; rather, it typically reflects that, for some nodes or iterations, the candidate splits do not improve the loss sufficiently. In practice, these warnings are often mitigated by tuning tree complexity controls (e.g., `min_child_samples`, `min_split_gain`, `num_leaves`) and by ensuring that constant or near-constant engineered features are removed.

This set of experiments expands the modelling landscape beyond bagging-based trees and confirms that modern gradient boosting frameworks provide the strongest performance among the tested families. LightGBM (`regression_l1`) achieves the best MAE (1.794) and a simple OOF blend with Huber regression provides a small additional gain (MAE 1.791). Quantile LightGBM further extends the framework by producing uncertainty bands, which are useful for risk-aware forecasting even when improvements in point-MAE are marginal.

4.7 Single Holdout Evaluation with Multiple Random Seeds

Cross-validation provides a reliable estimate of generalization error but is computationally expensive when repeated many times during tuning. As a complementary validation strategy, a single train/validation split was used together with repeated random seeds. The aim of this experiment was to quantify the sensitivity of LightGBM performance to the randomness of the holdout split and to derive a robust training configuration for the final submission.

4.7.1 Data and Leakage-Safe Preprocessing

The experiment was conducted on the feature-engineered dataset consisting of $n = 7578$ training samples and $p = 109$ predictors, with a separate test set of 3247 samples. Rows with missing target values were removed prior to splitting. To prevent leakage, preprocessing was fitted only on the training portion of each split and then applied to the validation portion. Numerical variables were imputed using the median. Categorical variables were imputed using the most frequent category and then one-hot encoded with `handle_unknown="ignore"` to ensure that unseen categories at validation/test time do not break the pipeline. Scaling was not applied because tree models are invariant to monotonic rescaling.

4.7.2 Robust Stratification for a Regression Target

A practical difficulty in holdout evaluation is ensuring that the training and validation sets are comparable. Simple random splits can accidentally produce validation folds that over-represent certain seasons or temperature regimes, leading to unstable MAE estimates. For this reason, a robust stratification rule was implemented. If the feature `season` is present, it is used as the stratification label after collapsing rare categories into a single “rare” label whenever their frequency is below a minimum threshold. This avoids split failures caused by categories appearing only once (a known limitation of stratified splitting). If `season` is unavailable or too imbalanced, stratification defaults to binning the continuous target into quantile bins using `qcut`. This produces approximately balanced strata over the target distribution while remaining compatible with stratified splitting.

4.7.3 Model Configuration and Early Stopping

The base learner was LightGBM with an MAE-aligned objective (`objective="regression_l1"`). Training used a large upper bound on boosting rounds (`n_estimators = 50000`) and relied on early stop-

ping with patience of 300 rounds, measured on the validation set. This approach serves two roles simultaneously: it prevents overfitting and it provides an empirical estimate of the effective optimal number of boosting iterations for the given split. The hyperparameters were kept fixed to the strongest configuration identified previously, and only the train/validation split seed was varied.

4.7.4 Experiment: Five Seeds for Holdout Stability

The dataset was split into 80% training and 20% validation for each seed in $\{0, 42, 123, 777, 2024\}$ using the robust stratification labels described above. For each split, the model was trained with early stopping and evaluated by MAE on the held-out validation set. Table 4.5 reports the results.

Seed	Holdout MAE	Best Iteration (early stopping)
42	**1.75910**	3796
2024	1.76826	851
0	1.77917	2510
777	1.78335	1183
123	1.81826	1325

Table 4.5: Holdout MAE and early-stopping iteration across five random seeds (80/20 split, stratified). Lower MAE is better.

Across these five randomized splits, the mean and standard deviation of MAE were

$$\mathrm{MAE}_{\text{mean}} \pm \mathrm{MAE}_{\text{std}} = 1.78163 \pm 0.02257,$$

indicating moderate sensitivity to the exact holdout partition. The best observed split occurred at seed 42 (MAE $= 1.75910$), with an early-stopping optimum of 3796 boosting rounds. This result also suggests that the model can reach substantially better error levels than the earlier 5-fold CV estimate (approximately 1.794), because a single holdout split can be easier (or harder) than the average CV fold. Therefore, this experiment is best interpreted as a stability diagnostic and a practical method for selecting a plausible iteration budget, rather than as a replacement for cross-validation.

4.7.5 Final Training and Submission Export

After selecting the best seed and its corresponding early-stopping iteration, the preprocessing pipeline was fitted on the full training dataset and the model was retrained on all $n = 7578$ observations using the chosen number of estimators (`n_estimators = 3796`). Predictions were then produced for the held-out competition test set and saved as a submission file.

Test identifiers were loaded from `test_ids.csv` when available (to preserve Kaggle's original ID ordering); otherwise, a fallback ID sequence $1, \ldots, N$ was used. The final predictions were exported to:

`submission_lgbm_5seed_best.csv`.

The resulting workflow provides a computationally efficient alternative to repeated cross-validation, yields a quantitative measure of split sensitivity, and offers a principled way to fix the iteration budget for the final training run.

4.8 Two-Stage Hyperparameter Tuning of LightGBM with Random Search (Stage A/B)

To obtain a competitive gradient-boosting model while keeping the search computationally feasible, a two-stage randomized hyperparameter tuning procedure was implemented for LightGBM (`LGBMRegressor`) using an MAE-aligned objective. The approach combines (i) 5-fold cross-validation with leakage-safe preprocessing, (ii) early stopping to estimate the effective number of boosting rounds.

4.8.1 Dataset and Preprocessing

All tuning experiments were performed on the feature-engineered training matrix `X_train_prepared.csv` and the associated target vector `y_train.csv`. The corresponding feature-engineered test matrix `X_test_prepared.csv` contains 3247 observations and the same feature schema.

A leakage-safe preprocessing pipeline was used within each cross-validation fold. Numerical variables were imputed using the median, while categorical variables were imputed using the most frequent category and one-hot encoded using `OneHotEncoder(handle_unknown="ignore")`. No scaling was applied because tree-based models are invariant to monotonic rescalings.

4.8.2 Cross-Validated MAE with Early Stopping

Each candidate hyperparameter configuration was evaluated with 5-fold shuffled K-Fold cross-validation (`random_state=42`). For each fold, the preprocessor was *fit only on the training split* and then applied to the validation split. The model used `objective="regression_l1"` to align optimization with the MAE metric. A large upper bound on the number of boosting rounds (`n_estimators=50000`) was set, and early stopping was used to prevent overfitting and to estimate a fold-specific optimal iteration count:

$$\texttt{early_stopping}(300),$$

meaning training stops if the validation MAE does not improve for 300 consecutive rounds. For each hyperparameter configuration, the experiment recorded the mean and standard deviation of MAE across folds, as well as the median of the fold-level best iteration numbers (`best_iter_median`). This median iteration count is later used as a stable estimate of the effective model complexity for training on the full dataset.

4.8.3 Resumable Experiment Design

A key practical issue with randomized hyperparameter search is runtime: evaluating many LightGBM configurations under cross-validation can take several hours, and interruptions are likely. To address this, the tuning code implements a resumable mechanism based on a stable hash key for each parameter dictionary.

For every sampled hyperparameter configuration, the parameter dictionary is serialized in a canonical form using JSON with sorted keys, and then hashed using MD5:

$$\texttt{param_key} = \texttt{md5}(\texttt{json.dumps(params, sort_keys=True)}).$$

All evaluated configurations are appended row-by-row to a CSV file together with their `param_key`. On restart, the script loads previously completed `param_key` values and skips those trials. This guarantees that re-running the script continues from the last unfinished configuration without duplication, while preserving a complete experimental log. Two independent resumable logs are maintained:

- `lgbm_tuning_stageA_coarse.csv` for Stage A (coarse search),
- `lgbm_tuning_stageB_refine.csv` for Stage B (refined search).

4.8.4 Stage A: Coarse Random Search

Stage A performs a broad exploration over a hand-designed discrete hyperparameter space covering learning rate, tree complexity, sampling, regularization, and histogram binning. Sixty configurations were sampled using `ParameterSampler` with a fixed seed (`random_state=42`), ensuring reproducibility. Each configuration was evaluated with the 5-fold CV procedure described above and appended to `lgbm_tuning_stageA_coarse.csv`.

At the end of Stage A, the CSV file is reloaded and sorted by mean CV MAE to identify the best coarse configuration. In the reported run, all 60 sampled configurations were detected as completed in the CSV file (confirming resumability), and the top configuration achieved a mean CV MAE of approximately 1.7775 with median best iteration around 9749 boosting rounds.

4.8.5 Stage B: Local Refinement Around the Stage A Best

Stage B refines the search by focusing on neighborhoods around the best Stage A parameters. For continuous or ordered discrete hyperparameters, a helper function `around()` selects a small set of values closest to the Stage A optimum. This narrows the search region while still allowing exploration of nearby alternatives. Fifty refined configurations were sampled (`random_state=7`) and evaluated using the same CV+early-stopping procedure, with results appended to `lgbm_tuning_stageB_refine.csv` in a resumable manner.

4.8.6 Best Configuration Identified (Stage A/B Result)

Stage B produced the best overall model found by the two-stage search. The best configuration and its cross-validated performance are reported below (lower MAE is better):

$$
\begin{aligned}
\texttt{learning_rate} &= 0.015,\\
\texttt{num_leaves} &= 127,\\
\texttt{max_depth} &= 14,\\
\texttt{min_child_samples} &= 25,\\
\texttt{min_child_weight} &= 0.001,\\
\texttt{subsample} &= 0.95,\\
\texttt{colsample_bytree} &= 0.55,\\
\texttt{reg_alpha} &= 0.0,\\
\texttt{reg_lambda} &= 0.05,\\
\texttt{min_split_gain} &= 0.002,\\
\texttt{max_bin} &= 511,\\
\texttt{extra_trees} &= \texttt{False},\\
\texttt{path_smooth} &= 0.0.
\end{aligned}
$$

The corresponding performance metrics were:

$$\mathrm{MAE}_{\mathsf{CV}} = 1.77728, \qquad \texttt{best_iter_median} = 8987.$$

These results indicate that a moderately deep model (depth 14) with relatively large leaf capacity (127 leaves) and strong subsampling (0.95 row sampling and 0.55 column sampling) provides the best trade-off between bias and variance under MAE evaluation.

4.8.7 Final Training

After selecting the best Stage B configuration, the preprocessing pipeline was refit on the full feature-engineered training dataset and applied to the feature-engineered test set. A final LightGBM model was then trained on all $n = 7578$ samples using `n_estimators` equal to the median early-stopping iteration count (8987). Test predictions were exported as a Kaggle-style submission file:

`submission_lgbm_tuned_stageAB.csv`.

In addition, both tuning logs were saved for reproducibility and auditability: `lgbm_tuning_stageA_coarse.csv`, `lgbm_tuning_stageB_refine.csv`.

4.8.8 Implementation Notes

During execution, `scikit-learn` emitted warnings of the form "X does not have valid feature names, but LGBMRegressor was fitted with feature names". This warning arises because the preprocessing step produces a transformed matrix without explicit column names (often a sparse matrix), while LightGBM

internally records feature names when fit on pandas objects. The warning does not affect numerical correctness of training or prediction. It can be suppressed by fitting LightGBM on the transformed matrices consistently (as done here) or by explicitly disabling feature-name checks, but it does not invalidate the reported results.

Overall, the Stage A/B procedure provides a reproducible and computationally manageable strategy for tuning a high-capacity boosting model, while the resumable design ensures robustness to long runtime and interruptions.

4.9 Multi-Horizon Chaining and Blended Ensembling (12h/24h/48h)

Beyond a single direct model for the 24-hour target, a complementary strategy is to exploit the *multi-horizon* structure of the dataset. In particular, since the dataset contains targets at $+12$h, $+24$h, and $+48$h, the 24-hour forecast can be expressed in two natural ways: as an update from the 12-hour forecast, or as a backward correction from the 48-hour forecast. This section describes a leakage-safe chaining and blending procedure that combines three predictors of the 24-hour temperature: (i) a direct $+24$h model, (ii) a 12-hour chained model, and (iii) a 48-hour chained model.

4.9.1 Predict 24h Directly and Indirectly via Deltas

Let y_{12}, y_{24}, and y_{48} denote the true targets at horizons 12h, 24h, and 48h. The method constructs two "delta" targets:

$$\Delta_{12\to 24} = y_{24} - y_{12}, \qquad \Delta_{24\to 48} = y_{48} - y_{24}.$$

Instead of relying only on a direct model for y_{24}, the approach also trains models for y_{12} and y_{48} and for the deltas. This yields two chained estimators of y_{24}:

$$\hat{y}_{24}^{(12)} = \hat{y}_{12} + w_{12}\,\widehat{\Delta}_{12\to 24}, \qquad \hat{y}_{24}^{(48)} = \hat{y}_{48} - w_{48}\,\widehat{\Delta}_{24\to 48},$$

where w_{12} and w_{48} are scalar weights in $[0,1]$ tuned to minimize MAE. The scalars are introduced because the delta models may be systematically over- or under-confident; allowing a learned shrinkage factor makes the chaining more robust. Finally, the chained predictions are blended with the direct 24-hour model:

$$\hat{y}_{24}^{\text{final}} = a\,\hat{y}_{24}^{\text{direct}} + b\,\hat{y}_{24}^{(12)} + c\,\hat{y}_{24}^{(48)}, \qquad a,b,c \geq 0,\;\; a+b+c=1,$$

where (a,b,c) are tuned on out-of-fold predictions.

4.9.2 Data Alignment and Target Availability

Because the feature-engineered design matrix is stored separately (`X_train_prepared.csv`) while targets are stored in the raw training file (`train.csv`), the script performs explicit row alignment. First, the rows are restricted to those where the 24-hour target is available, matching the standard workflow used elsewhere:

$$\mathcal{I}_{24} = \{i : \; y_{24,i} \text{ is not NaN}\}.$$

Then, to enable chaining through both 12h and 48h, the remaining dataset is further restricted to rows where both y_{12} and y_{48} are also available:

$$\mathcal{I} = \{i \in \mathcal{I}_{24} : \ y_{12,i} \text{ and } y_{48,i} \text{ are not NaN}\}.$$

All models in this experiment are trained on the resulting aligned subset. This is essential: without the intersection mask, the delta targets cannot be computed consistently, and the OOF estimates would not correspond to the same samples.

4.9.3 Leakage-Safe OOF Predictions for Each Sub-Model

To tune w_{12}, w_{48} and the blend weights (a, b, c) without leakage, the script relies on out-of-fold (OOF) predictions. A single 5-fold shuffled K-Fold split (`random_state=42`) is used consistently for every sub-model. For each target $t \in \{y_{24}, y_{12}, y_{48}, \Delta_{12\to24}, \Delta_{24\to48}\}$, the procedure is, for each fold:

1. fit preprocessor on training split
2. fit LightGBM on training split
3. predict validation split

This produces leakage-free OOF vectors: $\hat{\mathbf{y}}_{24}^{\text{direct}}$, $\hat{\mathbf{y}}_{12}$, $\hat{\mathbf{y}}_{48}$, $\widehat{\mathbf{\Delta}}_{12\to24}$, $\widehat{\mathbf{\Delta}}_{24\to48}$. Early stopping is used within each fold (`early_stopping(300)`) to obtain a fold-specific optimal iteration count; the script stores the median of those best iterations as a stable training budget for the final full-data fit.

Preprocessing is performed per-fold with median imputation for numerical features and mode imputation plus one-hot encoding for categorical features. No feature scaling is applied. This is the same leakage-safe strategy used throughout the boosting experiments.

4.9.4 Tuning the Chaining Weights

With OOF predictions in hand, the chaining weights are tuned using a simple grid search over $w \in [0, 1]$ with 201 values. For the 12h chain, the tuned prediction is:

$$\hat{\mathbf{y}}_{24}^{(12)}(w) = \hat{\mathbf{y}}_{12} + w\,\widehat{\mathbf{\Delta}}_{12\to24}.$$

The chosen w_{12} minimizes $\mathrm{MAE}(y_{24}, \hat{y}_{24}^{(12)}(w))$. An analogous procedure is used for w_{48} with

$$\hat{\mathbf{y}}_{24}^{(48)}(w) = \hat{\mathbf{y}}_{48} - w\,\widehat{\mathbf{\Delta}}_{24\to48}.$$

This step calibrates the indirect routes to y_{24} and prevents the delta models from injecting excessive variance.

4.9.5 Tuning the Final 3-Way Blend

After fixing w_{12} and w_{48}, the method tunes the final convex combination of three predictors:

$$\hat{\mathbf{y}}_{24}^{\text{final}}(a,b) = a\,\hat{\mathbf{y}}_{24}^{\text{direct}} + b\,\hat{\mathbf{y}}_{24}^{(12)} + (1-a-b)\,\hat{\mathbf{y}}_{24}^{(48)},$$

subject to $a \geq 0$, $b \geq 0$, and $a + b \leq 1$. A grid over $a, b \in \{0, 0.01, \ldots, 1\}$ is evaluated, and the pair (a, b) minimizing OOF MAE is selected. This produces a stable estimate of ensemble weights that reflect how much each route contributes to generalization performance. For transparency and diagnostics, the script saves the OOF components and the final blended OOF prediction to:

`oof_blend_3way_diagnostics.csv`.

This file can be used to inspect residual distributions, correlations between sub-models, and whether the blend improves systematically across seasons or temperature regimes.

4.9.6 Final Training and Submission Export

After selecting w_{12}, w_{48}, and (a, b, c) from OOF tuning, the script refits the preprocessing pipeline on the full aligned dataset and trains five LightGBM models on all available data:

$$y_{24}, \quad y_{12}, \quad y_{48}, \quad \Delta_{12\to 24}, \quad \Delta_{24\to 48}.$$

Each model uses the corresponding median best-iteration estimate (with a lower bound of 2000 estimators in the code) and then predicts the feature-engineered test set. The test predictions are combined using the same chaining and blend weights to yield $\hat{y}_{24}^{\text{final}}$ for the test set.

Finally, the method exports a Kaggle-style submission file:

`submission_blend_direct24_chain12_chain48.csv`.

Test identifiers are read from `test_ids.csv` when available; otherwise, a fallback $1, \ldots, N$ index is used to ensure that submission formatting is always valid.

This multi-horizon chaining approach is a form of structured ensembling that leverages the fact that forecasting horizons are not independent. In principle, the 12h and 48h tasks provide auxiliary supervision that can regularize the 24h prediction by forcing the model family to learn consistent temporal increments. The use of OOF tuning is essential: without it, using model predictions as inputs would introduce leakage and overly optimistic error estimates.

However, the approach also introduces additional complexity and potential instability, because errors can propagate through the chained routes (e.g., an error in $\hat{y}_{12}$ affects $\hat{y}_{24}^{(12)}$). The shrinkage weights (w_{12}, w_{48}) and the final convex blending weights (a, b, c) are therefore crucial components: they act as a variance control mechanism and determine whether chaining provides a net benefit relative to the direct 24-hour model alone.

This study set out to predict the 24-hour-ahead near-surface air temperature in Bern from a heterogeneous network of Swiss meteorological stations, under a setting that explicitly requires *spatial generalization* (Bern is not present in the training station set). Within this context, the experiments consistently showed that model families differ sharply in their ability to exploit the available structure.

First, classical linear and bagging-based tree models (Random Forest / Extra Trees) provided useful baselines but did not close the gap to state-of-the-art accuracy. In particular, feature engineering had little or even slightly negative impact for bagging ensembles, which is consistent with their invariance to monotone transformations and their reliance on averaging rather than sequential error correction. By contrast, boosting-based models benefited from the engineered representation and achieved the strongest performance overall, confirming that the target depends on nonlinear interactions, threshold effects, and regime-dependent behavior that are difficult to capture with purely additive or bagging-based methods. :contentReference[oaicite:0]index=0

Second, to obtain a competitive LightGBM configuration without an exhaustive search, a two-stage (Stage A/B) *resumable randomized tuning* strategy was implemented. Stage A explored a broad hyper-parameter space via random sampling and 5-fold cross-validation with leakage-safe preprocessing and early stopping; Stage B then refined the search locally around the best Stage A configuration. Beyond improving accuracy, the design choice of logging each configuration using a stable hash key and appending results to CSV files made the search robust to long runtime and interruptions, while preserving a complete, auditable experimental record. The best configuration found in Stage B achieved a mean cross-validated MAE of

$$\mathrm{MAE}_{\mathrm{CV}} = 1.77728,$$

with a stable effective iteration budget estimated by the median early-stopping iteration,

$$\texttt{best_iter_median} = 8987,$$

and was subsequently refit on the full training set to generate the final submission file

$$\texttt{submission_lgbm_tuned_stageAB.csv}.$$

Overall, this Stage A/B procedure offered an effective bias–variance trade-off (moderately deep trees with substantial leaf capacity and controlled sampling/regularization) and produced one of the strongest single-model results in the pipeline.

Finally, the multi-horizon chaining and blending strategy (leveraging the availability of +12h and +48h targets in training) provides a principled way to inject complementary information into the 24h forecast. By learning separate models for the direct 24h target and for inter-horizon deltas (12→24 and 24→48), and by tuning blending weights using leakage-safe out-of-fold predictions, the method reduces the risk of overfitting while enabling a controlled ensemble of partially independent estimators. Conceptually, the approach exploits consistency constraints across horizons: a 24h forecast can be reconstructed either by advancing the 12h forecast using a learned delta, or by stepping back from the 48h forecast using another learned delta. When these reconstructions are blended with the direct 24h model, the ensemble can correct idiosyncratic errors of any single pathway and improve robustness under station shift.

In summary, the empirical evidence supports three main conclusions: (i) the forecasting task is fundamentally nonlinear and is best addressed by boosting-based learners rather than linear or bagging ensembles; (ii) a structured, resumable Stage A/B randomized search provides a computationally feasible route to strong LightGBM configurations while maintaining reproducibility; and (iii) exploiting the multi-horizon structure via delta-based chaining and out-of-fold blending offers an additional mechanism to improve stability and potentially accuracy by combining complementary predictive signals.

4.10 Neural Network Baselines: MLP and 1D-CNN on Tabular Features

To complement the tree-based ensemble baselines, two neural network models were evaluated using PyTorch: a multilayer perceptron (MLP) and a simple one-dimensional convolutional neural network (1D-CNN) adapted to tabular data. The goal of this experiment was not to outperform gradient boosting, but to provide a modern deep-learning reference point under the same leakage-safe evaluation protocol and the same MAE metric.

4.10.1 Leakage-Safe Preprocessing and Cross-Validation Protocol

Both neural models were trained and evaluated using 5-fold shuffled cross-validation with a fixed random seed. To prevent data leakage, preprocessing was fitted independently inside each training fold and then applied to its corresponding validation fold. Numerical features were median-imputed and standardized (zero mean, unit variance), while categorical features were mode-imputed and one-hot encoded using `handle_unknown="ignore"`. The transformed design matrix can be sparse due to one-hot encoding; for neural training it was converted to a dense `float32` array.

To support statistical analysis and future ensembling, out-of-fold (OOF) predictions were stored for each model in separate files: `oof_predictions_mlp.csv` and `oof_predictions_cnn.csv`. Each file contains the true target `y_true` and the model-specific OOF predictions.

4.10.2 MLP Architecture and Training

The MLP baseline uses a fully-connected feedforward architecture with three hidden layers of sizes $(512, 256, 128)$. Each hidden layer is followed by batch normalization, ReLU activation, and dropout ($p = 0.2$). The network outputs a single scalar prediction. Training directly optimizes the MAE objective by using an ℓ_1 loss:

$$\mathcal{L}(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^{n} |\hat{y}_i - y_i|.$$

Optimization is performed with Adam (lr $= 10^{-3}$, weight_decay $= 10^{-5}$). A learning-rate scheduler (`ReduceLROnPlateau`) reduces the learning rate when validation MAE plateaus. Early stopping with patience of 20 epochs is applied to limit overfitting and reduce unnecessary computation.

Across 5-fold CV, the MLP achieved:

$$\mathrm{MAE}_{\mathsf{MLP}} = 1.86126 \pm 0.01631.$$

4.10.3 Tabular 1D-CNN Architecture and Training

A second neural baseline was implemented as a 1D convolutional network applied to the tabular feature vector. The feature vector is treated as a one-dimensional "signal" of length equal to the number of transformed input features. The model applies stacked 1D convolutions, ReLU activations, max-pooling, and dropout, followed by a small fully-connected head.

This approach is best interpreted as a heuristic baseline rather than a principled inductive bias for tabular meteorological predictors. Unlike images or time series, the ordering of one-hot-expanded tabular features is not physically meaningful, so convolutional locality assumptions are weak. Nevertheless, testing this architecture provides an informative negative control: if local feature grouping were beneficial, convolution could improve generalization. In practice, the 1D-CNN underperformed substantially:

$$\mathrm{MAE}_{\mathsf{CNN}} = 2.14069 \pm 0.04950.$$

The higher mean error and larger standard deviation indicate both worse accuracy and less stable generalization across folds. Table 4.6 summarizes the neural baselines.

Model	**CV MAE (mean)**	**CV MAE (std)**
MLP (512–256–128, MAE loss)	1.86126	0.01631
1D-CNN on tabular vector	2.14069	0.04950

Table 4.6: Neural network baselines evaluated with 5-fold cross-validation on the feature-engineered dataset. Lower MAE is better.

4.10.4 Why Gradient Boosting Performs Better

Despite the expressive capacity of neural networks, gradient boosting models (particularly LightGBM with an MAE-aligned objective) achieved clearly superior accuracy in this task. For reference, the tuned LightGBM configuration reached a cross-validated MAE of approximately 1.78, which is substantially lower than the best neural baseline (MLP at 1.86).

This performance gap is expected for several reasons.

1. Gradient boosting decision trees are extremely effective for heterogeneous tabular data, especially when feature interactions are primarily threshold-based and piecewise nonlinear. Meteorological predictors often contain regime changes (e.g., wind direction shifts, cloud-cover thresholds, precipitation onset), which are naturally captured by tree splits.

2. Boosting performs sequential error correction: each new tree targets residual structure left by the previous ensemble. This makes gradient boosting highly sample-efficient for structured tabular problems and well matched to MAE optimization.

3. Neural networks typically require either substantially larger datasets, stronger inductive structure (e.g., spatial grids, temporal sequences), or careful architecture design (embeddings for categoricals, normalization strategies, specialized tabular nets) to consistently outperform tree ensembles. In the present setting, one-hot encoding yields a sparse, high-dimensional input space, which increases optimization difficulty for neural models and can reduce effective signal-to-noise ratio. The CNN baseline further suffers because convolution assumes locality and translation structure that do not exist in one-hot tabular layouts.

Overall, the neural experiments provide useful baselines and confirm that, for this feature-engineered tabular forecasting problem and dataset size, modern gradient boosting methods remain the most effective approach in terms of both accuracy and stability.

5 Conclusion

Conclusion This paper focused on the task of short-term forecasts of air temperature in the city of Bern based on a diverse set of weather measurements obtained from a variety of stations across Switzerland. The gist of the task as a whole is in the problem of spatial generalization: the area in question is not explicitly presented in the training set. On the whole, the error value (MAE) was used as the major criterion to be evaluated. This choice reflects not only the goal of the competition but also the relevance to the problem.

As made clear by the experimental outcomes, model capacity and inductive bias are crucial in such a context. Traditional linear models, as much as they are interpretable and computationally light, are fundamentally flawed by their additive nature. While robust models like LAD and Huber regressions alleviated issues of instability and outliers, even the best setting of linear models trivially outperformed nonlinear models by a significant margin. Tree ensembles that used bagging (namely, Random Forest and Extra Trees) offered superior baselines, further supporting the existence of nonlinear interactions, and at the same time, their performance was utterly unaffected by feature engineering, as well as any further attempts to optimize. This correlates well with the theoretical characteristics of bagging algorithms, as they are, as they solely alleviate variance by averaging and, in effect, lack any capacity for error correction over time.

By contrast, tree models using boosting as their basis regularly produced significantly stronger results overall. Histogram Gradient Boosting and, more specifically, LightGBM with MAE-oriented objective function, worked considerably better in comparison with both linear and bagging models. Feature engineering in these models turned out to be a non-negligible aspect, which, in turn, confirms that boosters can process engineered features, interaction, or difference predictors efficiently. The two-step (A/B, or Stage) randomized search of regularized LightGBM's hyperparameters worked as an optimizing tool, providing a well-regularized LightGBM setting with a cross-validated approximate MAE of 1.78. The structured, two-step, and resumable nature of this,"optimize-as-you-go" process ensured computational efficiency, as well as traceability of experiments.

In addition to seeking to optimize individual models, the report also examined the application of multi-horizon learning as a means to add further structure to the forecasting problem. It was possible to combine the forecasting of the 12-hour, 24-hour, and 48-hour forecasts together, and the learning of the deltas between horizons, to effectively combine the models in a chained forecast for the 24-hour target. While the models were not particularly impressive in their additive benefits of combining, the method helped to increase robustness to individual error, and is useful in the presence of domain shift in the spatial distribution. In general, the empirical findings make it possible to form three major conclusions. First, in a generalized spatial context, the task of temperature prediction on the short timescale proves to be nonlinear in nature, and boosting-based models perform very well on this task. Second, proper hyper-parameter tuning, if assisted by a proper and resumable search methodology, has the potential to bring significant improvement on its own merit, even in absence of complete tuning searches. Third, multi-horizon models along with blending are an effective methodology to bring stability on the task, regardless of forecasts on individual time-horizons.

Appendix

A Use of Generative AI

Generative AI tools (primarily OpenAI's ChatGPT) were used exclusively to improve the clarity and grammatical correctness of the written text. All statistical thinking and interpretation of results, and substantive conclusions were carried out independently by the authors. AI tools were also used in several parts of the code to correct dysfunctional chunks of code. Finally, Generative AI was used to format tables using LaTeX code.

B Acknowledgements

This report was written based on our prior knowledge, the material covered during the lectures, and the textbook *The Elements of Statistical Learning*.